Measuring Differential Vulnerability to Policy Ambiguity Across LLM Agent Families

Anonymous

Abstract

Policy documents governing LLM agent behavior are written in natural language and inevitably contain ambiguities, yet the safety impact of such ambiguities remains unquantified. We present the first causal, type-granular, cross-model study of how policy ambiguity drives safety violations in LLM agents. Using a matched-pair design with perturbation control across 2,997 episodes, 5 architecturally diverse models, and 6 ambiguity types, we establish three findings. First, ambiguity causes a +34 percentage-point increase in safety violations (OR=15.1, p <0.0001), confirmed across all models and types. Second, a significant type hierarchy emerges (χ^2=28.02, p=3.6×10^{-5}): incompleteness triggers the highest violation rate (53.6%) while coreferential triggers the lowest (31.6%). Third, a model×type interaction reveals that authorization-scope vulnerability varies 52 percentage points across model families (12%–64%), while incompleteness remains universally dangerous (46%–62%). We release a benchmark of 300 clause pairs with matched-pair evaluation protocol. Our results provide a quantitative foundation for type-aware policy auditing and model-specific deployment risk assessment.

1 Introduction

Consider a retail return policy stating "items may be returned within 30 days." A customer returns a customized product—the agent grants a full refund because "items" scopes over all categories. The unambiguous variant specifying "standard items only" produces a correct refusal. One word of ambiguity costs $200 in unauthorized refunds.

As LLM agents are deployed in customer service, healthcare, and finance, their behavior is governed by natural-language policies that inevitably contain ambiguities (Berry and Kamsties, 2003; Massey et al., 2014). Prior work classifies ambiguity types in requirements (Li et al., 2024; Massey et al., 2014) and evaluates agent safety (Ruan et al., 2024; Yuan et al., 2024), but no study has *causally quantified* whether—and how much—different ambiguity types drive safety violations.

Berry and Kamsties (2003) distinguish *nocuous* ambiguity (causing divergent interpretations) from *innocuous* ambiguity (where context resolves the intended reading). In human requirements engineering, shared organizational context renders most ambiguity innocuous. LLM agents lack such shared context. Are policy ambiguities nocuous for agents? Does the *type* of ambiguity matter? Does it depend on the *model*?

We address these questions through a matched-pair experimental design on the τ^2-bench agent evaluation platform (Yao et al., 2024), testing 5 architecturally diverse models across 6 ambiguity types with perturbation control. Our contributions are:

1. **Causal quantification**: A +34 percentage-point increase in safety violations (OR=15.1) with perturbation control confirming ambiguity-specific causation (Δ_{perturb}=−0.67pp).
2. **Model×type interaction**: Authorization-scope vulnerability ranges 52pp across models (12% to 64%), while incompleteness is universally dangerous (46–62%). Safety training differentially mitigates specific ambiguity types.
3. **Benchmark and protocol release**: 300 clause pairs across 6 types with matched-pair evaluation design, enabling reproducible measurement of ambiguity vulnerability.

2 Related Work

Ambiguity Taxonomies. Li et al. (2024) introduce DRIP-R with 5 ambiguity types and measure inter-model interpretation divergence but not

safety violations. Massey et al. (2014) identify 6 types in privacy policies. Chantree et al. (2006) pioneer computational nocuous ambiguity detection in requirements. Ferrari et al. (2017) extend NLP-based detection across domains. In linguistics, the nocuous/innocuous distinction (Berry and Kamsties, 2003), good-enough processing (Ferreira et al., 2002), and garden-path phenomena (Frazier and Rayner, 1982) inform how readers process ambiguity. We bridge these traditions by measuring the *safety consequences* of each type.

Agent Safety Evaluation. ToolEmu (Ruan et al., 2024) and R-Judge (Yuan et al., 2024) evaluate agent safety, while InjecAgent (Zhan et al., 2024) studies prompt injection attacks. These works measure safety under adversarial or default conditions; none isolate ambiguity as a causal variable. Zhou et al. (2023) and Sclar et al. (2024) study instruction-following robustness and prompt sensitivity, but do not connect to ambiguity typology or safety outcomes.

Policy Specification Quality. PhantomPolicy (Li et al., 2025) and PolicyBank study policy completeness and safety, but neither controls for ambiguity type. Willis et al. (2008) and Ezzini et al. (2022) develop automated ambiguity detection in requirements. Our work is complementary: we measure the downstream *impact* of the ambiguities these tools detect.

3 Ambiguity Taxonomy

We adopt a two-layer, six-type taxonomy grounded in Li et al. (2024) and Massey et al. (2014).

Specification Layer. Ambiguities arising from incomplete or imprecise policy specification, lacking external disambiguation channels:

- **Authorization scope**: Vague permission or role boundaries (e.g., "authorized personnel").
- **Conditional precedence**: Conflicting rules without priority ordering.
- **Incompleteness**: Missing conditions, exceptions, or boundary cases.

Linguistic Layer. Ambiguities arising from natural-language structure, partially resolvable via context:

- **Lexical**: Polysemous or context-dependent word meaning.
- **Coreferential**: Ambiguous pronoun or relative-clause attachment.
- **Scopal**: Quantifier, negation, or modal scope interactions.

Each type has an operational definition with positive and negative examples, enabling reproducible clause generation. The specification/linguistic distinction parallels the pragmatics/semantics divide: specification ambiguity forces agents to *guess* when information is absent, while linguistic ambiguity may be partially resolved through tool responses or world knowledge.

Type	Exist.	New	Total	Pass	Bnd.
auth_scope	0	50	50	46	3
cond_prec	50	0	50	49	0
incompleteness	30	20	50	47	2
lexical	30	20	50	48	2
coreferential	0	50	50	45	1
scopal	50	0	50	49	0
Total	160	140	300	284	8

Table 1: Clause generation summary. Pass = quality check pass rate (93% overall). Bnd. = boundary overlap cases between types (2.7%, within tolerance).

4 Method

4.1 Clause Generation and Validation

We generated 300 matched clause pairs (ambiguous + unambiguous variants) across 6 types (50 per type), building on 160 existing clauses and creating 140 new ones. Table 1 summarizes the generation process.

Each pair consists of an ambiguous clause and a minimally modified unambiguous variant. Unambiguous variants add explicit conditions to resolve the ambiguity (mean disambiguation tokens: 35.2 for specification-layer, 18.4 for linguistic-layer). Quality validation followed a Chantree-style review process; 21 clauses required revision before passing. Eight clauses (2.7%) were flagged as boundary cases between types (5 authorization-scope/incompleteness, 3 coreferential/lexical).

4.2 Pilot Validation

Before the full study, we ran a mini-pilot on the two newly introduced types (authorization scope, coreferential): 5 clauses/type $\times$ 2 conditions $\times$ 1 model (GPT-5.4) = 20 episodes. Authorization scope showed a 40% ambiguous vs. 10% unambiguous violation rate; coreferential showed 20%

Metric	Value	Threshold	Verdict
Human inter-annotator κ	0.762	$\geq$0.75	PASS
Human–LLM pooled κ_w	0.807	$\geq$0.70	PASS
Human–LLM binary κ	0.740	$\geq$0.70	PASS
Cross-judge κ_w	0.685	$\geq$0.60	PASS
Same-family bias	None	None	PASS

Table 2: Judge calibration against human ground truth (N=100). All metrics pass pre-registered thresholds. κ_w = weighted Cohen's kappa. Ranking preservation ρ=0.70.

vs. 0%. Both types produced non-zero ambiguity effects, clearing the pilot threshold.

4.3 Experimental Design

We use a matched-pair within-clause design on τ^2-bench (Yao et al., 2024). Each of 300 clause pairs is tested under both conditions (ambiguous, unambiguous) across 5 models, yielding 2,997 episodes (3 dropped due to API timeouts).

Models. Three proprietary API models: GPT-5.4, GPT-4.1, Claude Sonnet 4.6; two open-source models via OpenRouter: Qwen3-235B (MoE), DeepSeek-V3 (dense). This spans three providers, two architecture types, and frontier to mid-tier capability levels.

Evaluation. Violations are judged by a cross-family LLM judge: GPT-5.4 is judged by GPT-4.1, GPT-4.1 by GPT-5.4, Claude by GPT-5.4, and open-source models by GPT-4.1. No model judges its own family.

4.4 Judge Calibration

We validated the LLM-as-judge pipeline against human ground truth on 100 stratified episodes annotated by 2 human raters (Table 2).

Human–LLM agreement (κ_w=0.807) is substantial, confirming that the violation measurement chain is trustworthy. No same-family judging bias was detected.

4.5 Realism Validation

We compared the distributional properties of our synthetic clauses against naturally occurring ambiguities from DRIP-R (Li et al., 2024) using Kolmogorov–Smirnov tests on four linguistic metrics (Table 3).

Synthetic clauses are $\sim$5 tokens longer and $\sim$0.7 parse levels deeper than natural ambiguities, a consequence of the matched-pair design requiring explicit disambiguation. Readability and lexical diversity are not significantly different. We note that our perturbation control (§4.6) provides stronger causal validity than distributional similarity.

Metric	Synth.	DRIP-R	p	Sig?
Sentence length	31.4	26.1	.028	Yes
Flesch–Kincaid	12.8	12.2	.112	No
Parse depth	4.8	4.1	.039	Yes
Type-token ratio	0.71	0.73	.234	No

Table 3: Realism validation: synthetic vs. DRIP-R clauses. Two of four metrics show significant distributional differences, attributable to the matched-pair design requiring longer disambiguation text.

Metric	Value	Threshold
Δ_{perturb} (pp)	-0.67	< 5
95% CI upper bound	6.67	< 10
Ratio (Δ_{perturb} / Δ_{amb})	0.02	< 0.15
McNemar p	1.0	—

Table 4: Perturbation control results. Non-ambiguity surface changes produce Δ=$-$0.67pp (CI contains zero), compared to the ambiguity Δ=+34pp. PASS on all thresholds.

4.6 Perturbation Control

To distinguish ambiguity-specific effects from general prompt brittleness, we tested 30 clause pairs with non-ambiguity surface changes (synonym substitution, clause reordering) across 300 episodes (Table 4).

Non-ambiguity perturbations produce near-zero effect (Δ=$-$0.67pp, CI [$-$8.0, 6.67]), confirming that the observed +34pp effect is ambiguity-specific, not an artifact of prompt sensitivity. The ratio of 0.02 means perturbation effects account for less than 2% of the ambiguity effect.

4.7 Test–Retest Reliability

To assess measurement stability, we tested 30 clauses $\times$ 3 replicates $\times$ 2 conditions $\times$ 2 models = 360 episodes (Table 5).

Overall ICC=0.919 (excellent) confirms that per-clause violation rates are stable across independent runs. The layer pattern is consistent: linguistic-layer types (ICC 0.976–0.989) are more reproducible than specification-layer types (ICC 0.807–0.857), reflecting the determinism of disambiguation channels.

Type	Layer	ICC	95% CI
scopal	Linguistic	.989	[.97, .99]
lexical	Linguistic	.981	[.96, .99]
coreferential	Linguistic	.976	[.95, .99]
auth_scope	Specification	.857	[.74, .93]
cond_prec	Specification	.856	[.74, .93]
incompleteness	Specification	.807	[.67, .90]
Overall	—	**.919**	[.89, .94]

Table 5: Test–retest ICC by type. Overall ICC=0.919 (excellent). Linguistic-layer types show higher reproducibility than specification-layer types, consistent with the disambiguation channel framework.

Model	Amb	Unamb	Δpp	p
GPT-5.4	.410	.027	+38.3	$<10^{-17}$
GPT-4.1	.407	.060	+34.7	$<10^{-17}$
DeepSeek-V3	.497	.107	+39.0	$<10^{-17}$
Qwen3-235B	.431	.007	+32.4	$<10^{-17}$
Claude S. 4.6	.303	.047	+25.7	$<10^{-17}$
Pooled	.410	.045	**+34.0**	<.0001

Table 6: C1: Binary ambiguity effect by model. Pooled OR=15.1 [10.78, 21.17]. Cross-judge κ=0.69. All models show highly significant effects. Logistic regression: condition coefficient=2.278, z=19.65.

5 Results

5.1 C1: Binary Effect

Ambiguity causes a massive, universal increase in safety violations across all 5 models (Table 6).

The pooled effect is +34.0pp (OR=15.1, 95% CI [10.78, 21.17], p <0.0001). Bootstrap CI for the pooled delta is [31.2, 36.9]pp. The effect holds for every model tested, ranging from +25.7pp (Claude) to +39.0pp (DeepSeek). Logistic regression confirms ambiguity as the dominant predictor (coefficient=2.278, z=19.65) even after controlling for token delta (β=0.022, p=0.0005), model, and type.

5.2 C2: Type Hierarchy

A significant type hierarchy emerges among the six ambiguity types (Table 7).

The omnibus test is significant (χ^2=28.02, p=3.6×10^{-5}, Cramér's V=0.137). While V=0.137 falls below the conventional 0.20 medium-effect threshold, statistical power is 0.993 with MDE=0.093, and the hierarchy is well-powered. A three-tier structure emerges from TOST analysis: incompleteness (53.6%) stands alone as most dangerous, four types cluster in the mid range (36.8–42.8%), and coreferential (31.6%) is lowest but still causes substantial violations (+18.7pp vs. unambiguous). At the ±10pp equivalence margin, only 4/15 type pairs are equivalent.

Type	Viol. Rate	95% CI	Tier
incompleteness	53.6%	[48.2, 59.0]	High
lexical	42.8%	[37.4, 48.2]	Mid
auth_scope	41.6%	[36.2, 47.0]	Mid
scopal	39.4%	[34.1, 44.7]	Mid
cond_prec	36.8%	[31.5, 42.1]	Mid
coreferential	31.6%	[26.5, 36.7]	Low

Table 7: C2: Type hierarchy (ambiguous condition, 5-model pooled). Omnibus χ^2=28.02, p=3.6×10^{-5}, V=0.137. Power=0.993, MDE=0.093.

Group	Spec	Ling	Δpp	p	Verdict
Pooled	44.0%	37.9%	+6.08	.017	Sig.
GPT-4.1 judged	48.4%	40.4%	+8.0	.059	Marg.
GPT-5.4 judged	41.0%	36.2%	+4.8	.157	N.S.

Table 8: C3: Layer effect. The pooled effect is significant but dissolves under within-judge analysis, suggesting it may partly reflect between-judge variance.

5.3 C3: Layer Effect

Specification-layer ambiguities cause moderately more violations than linguistic-layer ambiguities on average, but this effect is fragile under subgroup analysis (Table 8).

The pooled effect (Δ=6.08pp, p=0.017) is significant but small (V=0.062). Within-judge subgroups are non-significant (GPT-4.1 judged: p=0.059; GPT-5.4 judged: p=0.157), indicating that the pooled significance may partly reflect between-judge variance. We report C3 as *suggestive* rather than confirmed.

5.4 C4: Model × Type Interaction

Models show significantly different vulnerability profiles (Table 9). The model main effect is significant (χ^2=31.02, p=3.0×10^{-6}, V=0.102), and the model×type interaction reveals qualitatively distinct patterns.

Three patterns emerge:

- **Authorization scope** is the model-differentiating type: 52pp range from Claude (12%) to GPT-5.4 (64%). Claude's safety training appears to specifically address authorization boundary handling.
- **Incompleteness** is universally dangerous: 46–62% across all models (16pp range). No model's

Type	G5.4	G4.1	Cld	Qw3	DS	Rng
auth_scope	64	46	12	26	60	52
incomp.	46	58	48	62	54	16
lexical	42	34	32	52	54	22
coref.	18	32	22	42	44	26
scopal	42	38	32	37	48	16
cond_prec	34	36	36	40	38	6

Table 9: C4: Model $\times$ type violation rates (%, ambiguous). G5.4=GPT-5.4, G4.1=GPT-4.1, Cld=Claude, Qw3=Qwen3, DS=DeepSeek. Rng=max$-$min range (pp). Authorization scope shows the widest range (52pp).

safety training compensates for missing information.

- **Conditional precedence** is model-invariant: 34–40% (6pp range), reflecting that explicit rule conflicts are handled similarly by all architectures.

Per-model type differentiation varies: GPT-5.4 (χ^2=23.44, p=0.0003) and Claude (χ^2=17.87, p=0.003) show strong type-specific vulnerability profiles. DeepSeek shows no significant type differentiation (χ^2=6.31, p=0.277, V=0.14), exhibiting uniformly high vulnerability across all types. Inter-model agreement on which specific clauses trigger violations is only fair (Fleiss κ=0.384): the binary effect is universal, but *which* clauses are dangerous is model-dependent.

Open-Source Models. The two open-source models confirm C1 universality. Qwen3-235B (Δ=+32.4pp) shows moderate type differentiation (χ^2=15.89, p=0.007), with incompleteness (62%) and lexical (52%) as its most vulnerable types. DeepSeek-V3 (Δ=+39.0pp) shows the highest overall vulnerability (49.7%) with a uniformly high profile across all types.

6 Discussion

Differential Nocuity. Our results extend Berry and Kamsties' (2003) nocuous ambiguity framework to LLM agents. All six ambiguity types are nocuous (+34pp overall), but the *degree* of nocuity depends on both type and model. The model$\times$type interaction (C4) is the most practically consequential finding: deployment risk cannot be assessed from ambiguity type alone or model identity alone—it requires their intersection.

Systematic Nocuity. In 27% of ambiguous episodes, all models converge on the *same wrong* interpretation. This "systematic nocuity" is distinct from Berry and Kamsties' original divergent-interpretation framework and is arguably more dangerous: it is invisible in inter-model agreement metrics and appears correct until compared against ground truth.

Ambiguity Blindness. Exploratory automated analysis of 614 violation traces (§A) reveals that agents show very low awareness of ambiguity: only 4.1–10.7% of violation traces contain any acknowledgment that the policy clause is ambiguous. Over 88% of traces show surface adoption of the first plausible interpretation, and fewer than 1% escalate to the user. We note that this analysis uses LLM-based coding with moderate inter-coder agreement (κ=0.52); the binary awareness finding is robust, but fine-grained failure mode patterns should be interpreted cautiously (see Appendix B).

Implications for Model Selection. The 52pp authorization-scope range across models suggests that safety training quality varies substantially in its coverage of ambiguity types. Practitioners deploying agents in authorization-sensitive domains (banking, healthcare access control) should prioritize models with demonstrated robustness to authorization-scope ambiguity, while recognizing that *no* model is robust to incompleteness.

7 Limitations

Our study has three main limitations. First, the primary evaluation covers two domains (retail, airline) on τ^2-bench. Cross-benchmark replication on SOPBench (Appendix C) shows attenuated effects (+14.2pp vs. +34.0pp) with 2/5 models non-significant, suggesting effect size depends on domain and evaluation methodology. Second, the synthetic matched-pair design produces clauses that are detectably longer and more complex than naturally occurring ambiguities (2/4 KS tests significant, Table 3); our perturbation control provides causal validity but does not fully address ecological concerns. Third, automated failure mode coding (κ=0.22 against human re-annotation, Appendix B) does not support fine-grained mechanism claims; we therefore present failure mode analysis as exploratory.

8 Conclusion

We present the first causal, type-granular, cross-model study of policy ambiguity's impact on LLM

agent safety. All ambiguity types are nocuous (+34pp, OR=15.1), but type *and* model jointly determine risk level. Authorization-scope vulnerability varies 52pp across model families, while incompleteness remains universally dangerous. Our benchmark of 300 clause pairs with matched-pair evaluation protocol provides a quantitative foundation for type-aware policy auditing and model-specific deployment risk assessment.

References

Daniel M. Berry and Erik Kamsties. 2003. Ambiguity in requirements specification. In *Perspectives on Software Requirements*, pages 7–44. Springer.

Francis Chantree, Bashar Nuseibeh, Anne de Roeck, and Alistair Willis. 2006. Identifying nocuous ambiguities in natural language requirements. In *Proc. ASE*, pages 245–254.

Saad Ezzini, Sallam Abualhaija, Chetan Arora, Mehrdad Sabetzadeh, and Lionel C. Briand. 2022. TAPHSIR: Towards AnaPHoric ambiguity detection and resolution in requirements. In *Proc. ESEC/FSE*, pages 229–240.

Fernanda Ferreira, Karl G. D. Bailey, and Vittoria Ferraro. 2002. Good-enough representations in language comprehension. *Current Directions in Psychological Science*, 11(1):11–15.

Alessio Ferrari, Gloria O. Spagnolo, and Stefania Gnesi. 2017. Detecting requirements defects with NLP patterns. In *Proc. ASE*, pages 52–57.

Lyn Frazier and Keith Rayner. 1982. Making and correcting errors during sentence comprehension: Eye movements in the analysis of structurally ambiguous sentences. *Cognitive Psychology*, 14(2):178–210.

Daniel Lassiter and Noah D. Goodman. 2017. Adjectival vagueness in a Bayesian model of interpretation. *Synthese*, 194(10):3801–3836.

Zhaowei Li, Mairgup Mansur, and Kenny Q. Zhu. 2024. DRIP-R: Disambiguating references to imprecise policies in requirements. In *Proc. ACL*, pages 1234–1248.

Yang Li, Zeyu Zhang, and Chao Zhang. 2025. PhantomPolicy: Evaluating agent safety with phantom policies. *arXiv preprint arXiv:2501.XXXXX*.

Aaron K. Massey, Richard L. Rutledge, Annie I. Antón, and Peter P. Swire. 2014. Identifying and classifying ambiguity for regulatory requirements. In *Proc. RE*, pages 83–92.

Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. 2024. Identifying the risks of LM agents with an LM-emulated sandbox. In *Proc. ICLR*.

Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. 2024. Quantifying language models' sensitivity to spurious features in prompt design or: How I learned to start worrying about prompt formatting. In *Proc. ICLR*.

Alistair Willis, Francis Chantree, and Anne de Roeck. 2008. Automatic identification of nocuous ambiguity. *Research on Language and Computation*, 6(3):355–374.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*.

Tongxin Yuan, Zhiwei He, Lingzhong Dong, Yiming Wang, Ruijie Zhao, Tian Xia, Lizhen Xu, Binglin Zhou, Fangqi Li, Zhuosheng Zhang, Rui Wang, and Gongshen Liu. 2024. R-Judge: Benchmarking safety risk awareness for LLM agents. In *Proc. EMNLP*, pages 3614–3635.

Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Proc. ACL Findings*, pages 3962–3985.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. 2023. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*.

A Exploratory Error Analysis

We conducted automated error analysis of 614 violation traces using two LLM coders (GPT-5.4: 123 traces, GPT-4.1: 122 traces) with a 4-category failure mode taxonomy. Table 10 shows the LLM-coded dominant failure mode by ambiguity type; Table 11 shows per-type inter-coder agreement.

Type	G5.4 Dom.	%	G4.1 Dom.	%	Agr?
auth_sc.	unauth_esc	72	unauth_esc	83	Yes
incomp.	assum_act	61	assum_act	66	Yes
scopal	scope_mis	76	scope_mis	74	Yes
lexical	assum_act	71	assum_act	53	Part.
cond_pr.	scope_mis	41	arb_rule	28	No
coref.	scope_mis	44	assum_act	38	No

Table 10: LLM-coded dominant failure mode by type (exploratory).

Overall LLM–LLM κ=0.517 (moderate). The two LLM coders agree on the dominant failure mode for 3/6 types; all 6 have majority-level agreement. Disambiguation awareness is low (GPT-5.4: 4.1%, GPT-4.1: 10.7%), and surface adoption exceeds 88% across all types. However,

Type	N	% Agr.	κ	Issue
auth_scope	42	54%	.054	keyword bias
incomp.	56	62%	.233	fuzzy boundary
lexical	38	92%	.000	κ paradox
scopal	36	100%	—	perfect agr.
cond_prec	42	58%	.312	label overlap
coref.	31	48%	.184	flat distrib.

Table 11: Per-type inter-coder agreement. Overall LLM–LLM κ=0.517. The κ paradox inflates lexical agreement; scopal κ is undefined due to perfect agreement on a single label.

this analysis should be interpreted as exploratory due to the κ re-annotation findings described in Appendix B.

B Human Re-Annotation of Failure Modes

To validate the automated failure mode coding, we conducted a human re-annotation study on 25 traces using an improved codebook (Table 12).

Comparison	κ	Interpretation
Human–Human	0.48	Moderate
Human–LLM (pooled)	0.22	Poor
LLM–LLM (original)	0.52	Moderate (inflated)

Table 12: κ re-annotation summary. Human–LLM κ=0.22 indicates systematic LLM coding bias.

Type	LLM Dom.	%	Hum. Dom.	%	Agr?
auth_sc.	unauth_esc	78	scope_mis	52	No
incomp.	assum_act	64	inapp_act	45	No
scopal	scope_mis	75	scope_mis	68	Yes
lexical	assum_act	62	assum_act	41	No
cond_pr.	arb_rule	34	arb_rule	42	Yes
coref.	scope_mis	41	assum_act	36	No

Table 13: Human vs. LLM dominant failure mode by type. Agreement on only 2/6 types. LLM coders show keyword-driven label assignment (e.g., "unauthorized" $\rightarrow$ unauthorized_escalation).

The human–LLM κ=0.22 reveals that LLM coders rely on surface keyword matching rather than interpreting the underlying failure mechanism. The human–human κ=0.48 (moderate) indicates that the failure mode categories themselves have inherently fuzzy boundaries, making trace-based mechanism inference fundamentally challenging. Consequently, we present all failure mode findings as exploratory automated analysis.

Type	LLM Bias	Human Finding
auth_sc.	keyword $\rightarrow$ unauth_esc	40% are scope_mis
incomp.	keyword $\rightarrow$ assum_act	30% are inapp_act
lexical	κ paradox inflation	true diversity higher
coref.	scope_mis plurality	flat, no dominant

Table 14: Systematic LLM coding biases identified in re-annotation.

C SOPBench Cross-Benchmark Replication

We replicated C1 on SOPBench (Healthcare + Bank domains): 100 clause pairs $\times$ 2 conditions $\times$ 5 models = 987 episodes (13 dropped).

Model	Amb	Unamb	Δpp	p	Rep?
GPT-5.4	.312	.128	+18.4	.003	Yes
GPT-4.1	.288	.120	+16.8	.008	Yes
DeepSeek	.298	.148	+15.0	.015	Yes
Qwen3	.262	.136	+12.6	.061	No
Claude	.194	.112	+8.2	.093	No
Overall			**+14.2**		3/5

Table 15: SOPBench C1 replication. Overall Δ=+14.2pp (vs. +34.0pp on τ^2-bench). 3/5 models reach significance; Qwen3 and Claude do not.

Domain	Amb	Unamb	Δpp
Healthcare	.289	.128	+16.1
Bank	.253	.130	+12.3

Table 16: SOPBench by domain.

The effect replicates directionally in 3/5 models but with substantially attenuated magnitude (+14.2pp vs. +34.0pp). Type ranking correspondence with τ^2-bench is non-significant (ρ=0.20, p=0.80), suggesting the type hierarchy is domain-dependent. We attribute the attenuation to SOPBench's rule-based verifier capturing only explicit rule violations, whereas τ^2-bench's LLM judge detects nuanced safety violations.

D DeBERTa Classifier Proof-of-Concept

We fine-tuned DeBERTa-v3-large for binary ambiguity detection and per-type classification on our 300 clause pairs using 5-fold cross-validation.

Binary detection is serviceable (F1=0.78), but per-type classification (macro F1=0.55) is limited. The per-type difficulty gradient (conditional precedence easiest, incompleteness hardest) mirrors the annotation challenges identified in

Type	Prec.	Rec.	F1
cond_prec	.91	.87	.89
scopal	.74	.68	.71
coreferential	.65	.59	.62
auth_scope	.54	.48	.51
lexical	.38	.31	.34
incompleteness	.32	.25	.28
Binary (any)	.82	.74	.78
Macro (6-type)	—	—	.55

Table 17: DeBERTa classifier performance. Binary F1=0.78 (FPR=6.2%). Per-type macro F1=0.55. Difficulty gradient mirrors annotation challenges.

the κ re-annotation study, corroborating that fine-grained type boundaries are inherently fuzzy.